

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.6

Comprehensions

Day 6 — four lines of loop, written as one



Boston University

90 min

Concepts + live code

30 min

Practice notebook

3

Kinds: list, dict, set

Prepared & presented by **Srinivasa Sai Chava**

Before we start — five questions from Day 5

1 What does `d["missing"]` do?

→ Raises `KeyError`. Use `.get()` if the key is optional.

2 Is `88 in marks` `True` when `88` is a value?

→ No — `'in'` checks keys only.

3 Write the counting line.

→ `counts[k] = counts.get(k, 0) + 1`

4 What does looping a dict give you?

→ The keys. Use `.items()` for pairs.

5 Why can't a list be a dictionary key?

→ Keys must be immutable so their hash stays valid.

Yesterday I promised the filter loop would become one line. Today I deliver on that.

How today's two hours are structured

1

List comprehensions

30 min

The transformation, conditions, if/else

2

Dict & set comprehensions

20 min

The same idea, different brackets

3

Going further

20 min

Nesting, and generator expressions

4

When NOT to use one

10 min

Readability is the whole point

5

Mini build

10 min

Rewriting a data-cleaning script

6

Practice notebook

30 min

You code, I walk around

Section 4 is not filler. Comprehensions are the feature students most often overuse once they learn it.

By the end of today you will be able to

1

Translate

Turn any simple build-a-list loop into a comprehension, and back again

2

Filter

Keep only the items you want, with a condition

3

Transform

Build dictionaries and sets the same way, not just lists

4

Judge

Decide when a comprehension helps — and when a plain loop is better

Nothing today is new capability

Every comprehension you write today could be written as a loop you already know. What changes is that four lines become one, and a reader can take in the whole operation at a glance. Comprehensions are about clarity — so the moment one stops being clear, it has failed at its only job.

1

List Comprehensions

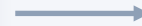
The transformation · 30 minutes

- Loop → one line
- The syntax
- Adding a condition
- if / else
- Scope

From loop to comprehension

The loop you already write

```
squares = []  
for n in numbers:  
    squares.append(n * n)
```



The same thing, as one line

```
squares = [  
    n * n for n in numbers]
```

`n * n`

WHAT to build — the expression, taken from the append() call

`n`

the loop variable — same name, same job

`numbers`

WHAT to loop over — the iterable

*Read it as: “build $n*n$, for each n in numbers.” The pieces are identical — only the order changed.*

The syntax, and what it replaces

```
[ expression for item in iterable ]
```

The square brackets say “build me a list”. Everything inside says what to put in it.

Square every number

```
[n * n for n in nums] → [1, 4, 9]
```

Uppercase every name

```
[n.upper() for n in names] → ['RAVI', 'SARA']
```

Length of each word

```
[len(w) for w in words] → [6, 2, 5]
```

Pull one field from rows

```
[r["name"] for r in rows] → ['Ravi', 'Sara']
```

Cast a column to int

```
[int(x) for x in row] → [88, 71, 64]
```

Just copy a list

```
[n for n in nums] → same values, new list
```

The last one is worth noticing: `[n for n in nums]` is exactly `nums.copy()` from Day 4 — the expression can simply be the item itself.

Adding a condition — keeping only some items

```
[ expression for item in iterable if condition ]
```

The if goes at the END. Items that fail the test are simply not included.

Loop version

```
passed = []
for m in marks:
    if m >= 40:
        passed.append(m)
```

Comprehension version

```
passed = [m for m in marks if m >= 40]
```

Four lines to one. The nested if became a trailing if — the shape of the loop is preserved exactly.

```
[n for n in nums if n % 2 == 0]
# even numbers only
```

```
[r for r in rows if r["mark"] >= 40]
# the rows that passed
```

```
[w for w in words if len(w) > 3]
# words longer than three letters
```

```
[n.strip() for n in raw if n.strip()]
# clean, and drop the blanks
```

if / else — the one that trips everybody

Filtering — if goes at the END

```
[n for n in nums if n > 0]
```

“Keep n, for each n, IF it is positive.”

Some items are dropped, so the result may be **SHORTER** than the input.

Choosing — if / else goes at the FRONT

```
["pos" if n > 0 else "neg" for n in nums]
```

“Build pos-or-neg, for each n.”

Nothing is dropped, so the result is **ALWAYS** the same length as the input.

You cannot put a bare if at the front

```
[n if n > 0 for n in nums]  
-> SyntaxError
```

At the front you are writing Day 1's conditional expression, and that always needs an else — Python must know what to build for every item. At the end you are filtering, and no else is possible.

The way to remember it: if at the end DROPS items. if/else at the front CHANGES items. **Ask which you want — a shorter list, or a different list — and the position follows.**

2

Dict & Set

Same idea, different brackets · 20 minutes

- Dict comprehensions
- Set comprehensions
- No tuple version

Dictionary comprehensions

```
{ key_expression : value_expression for item in iterable }
```

Curly brackets and a colon. The colon is what makes it a dict rather than a set.

Yesterday's filter, as promised

```
passed = {}  
for k, v in marks.items():  
    if v >= 40:  
        passed[k] = v
```

```
{k: v for k, v in marks.items() if v >= 40}
```

Four more you will actually use

```
{k: v*2 for k, v in d.items()} # scale values  
{v: k for k, v in d.items()} # INVERT it  
{w: len(w) for w in words}  # word -> length  
{n: n*n for n in range(1, 6)} # number -> square  
dict(zip(names, marks))      # two lists -> dict
```

The inversion line is worth pausing on. `{v: k for k, v in d.items()}` was a five-line loop in yesterday's practice — and it carries the same warning: if two keys share a value, one of them wins and the other is silently lost.

Set comprehensions — and the one that does not exist

Curly brackets, no colon

```
{n * n for n in [1, 2, 2, 3]}
-> {1, 4, 9}           duplicates dropped

{w.lower() for w in names}
-> unique lowercase names

{len(w) for w in words}
-> the distinct word lengths
```

⚠ There is no tuple comprehension

```
(n * n for n in nums)

-> NOT a tuple.
    This is a GENERATOR.
```

Round brackets make something different, not a tuple version of the same thing. If you want a tuple, build a list and convert it: `tuple([n*n for n in nums])`.

The three brackets, side by side

You write	You get	Because
[n for n in x]	a list	square brackets
{n for n in x}	a set	curly brackets, no colon
{k: v for k, v in x}	a dict	curly brackets WITH a colon
(n for n in x)	a generator	round brackets — see section 3

3

Going Further

Nesting and generators · 20 minutes

- Two loops in one
- Nested comprehensions
- Generator expressions

Two loops inside one comprehension

Flattening a nested list

```
flat = []
for row in matrix:
    for n in row:
        flat.append(n)
```

```
flat = [n for row in matrix for n in row]
```

The two for clauses appear in the SAME order as the nested loops — outer first, then inner.

The order feels backwards at first

The expression comes first, then the loops read left-to-right exactly as you would write them stacked. Many people instinctively reverse the two for clauses — and get a `NameError`, because the inner variable does not exist yet.

```
[n for n in row for row in matrix]
-> NameError: name 'row' is not
    defined
```

Building a grid — a comprehension inside a comprehension

```
table = [[i * j for j in range(1, 4)]
          for i in range(1, 4)]

# [[1,2,3], [2,4,6], [3,6,9]]
```

Note the difference: flattening uses two for clauses in ONE set of brackets and gives a flat list. This uses a whole comprehension as the expression, so each outer pass produces a complete inner list — and you get a list of lists.

Generator expressions — the lazy cousin

Round brackets instead of square

```
squares = [n*n for n in range(1000000)]  
-> builds a MILLION values in memory  
  
squares = (n*n for n in range(1000000))  
-> builds NOTHING yet  
    produces each value only when asked  
  
sum(n*n for n in range(1000000))  # brackets  
                                # optional here
```

When it is worth it

- You only need to pass through the data once
- You are feeding it straight into `sum()`, `max()`, `any()` or a for loop
- The data is large enough that holding it all in memory matters

⚠ A generator is used up after one pass

```
g = (n for n in [1, 2, 3])  
  
sum(g)    -> 6  
sum(g)    -> 0    ! it is exhausted
```

A list can be looped over as many times as you like. A generator hands out its values once and is then empty — with no error to warn you, just a wrong answer. Use a list unless you have a specific reason not to; today this is recognition-level knowledge.

4

When NOT To Use One

Readability is the whole point · 10 minutes

- The limits
- Side effects
- Scope

When a plain loop is the better answer

1 Too much logic packed in

```
[transform(x) if check(x) else  
fallback(x) for row in data  
for x in row if x and valid(x)]
```

Nobody can read this — including you, next month. Use a loop.

2 You only want a side effect

```
[print(n) for n in nums]  
  
# builds a list of None values  
# and throws it away
```

If you are not building a collection, write a for loop.

3 The body needs several steps

```
for row in rows:  
    row = clean(row)  
    if not row: continue  
    log(row)  
    out.append(row)
```

A comprehension holds ONE expression. This needs a loop.

The test: can you read it aloud in one breath and have someone understand it? If yes, the comprehension is helping. If you had to pause and work it out, write the loop — the loop is not a failure, it is the clearer choice. Two conditions or two for clauses is roughly the ceiling.

Debug corner — eight comprehension bugs

You wrote	What happens	Fix
<code>[n if n > 0 for n in x]</code>	SyntaxError	<code>[n for n in x if n > 0]</code>
<code>[n for n in x if n > 0 else 0]</code>	SyntaxError	<code>[n if n > 0 else 0 for n in x]</code>
<code>(n for n in x)</code>	a generator, not a tuple	<code>tuple([n for n in x])</code>
<code>{n for n in x}</code>	a set — duplicates dropped	<code>[n for n in x]</code>
<code>[print(n) for n in x]</code>	builds a list of None	use a plain for loop
<code>[n for n in row for row in m]</code>	NameError: 'row'	outer loop first
<code>sum(g) twice</code>	second call gives 0	use a list, not a generator
<code>{k for k, v in d.items()}</code>	a set of keys, no values	add the colon: <code>{k: v ...}</code>

Rows 1 and 2 are the same confusion from both sides: if at the END filters, if/else at the FRONT chooses. All eight are in today's practice notebook.

Putting it together — a cleaning script, rewritten

Before — 20 lines

```
names = []
for n in row:
    n = n.strip().title()
    if n:
        names.append(n)

book = {}
for i in range(len(names)):
    book[names[i]] = marks[i]

passed = {}
for k, v in book.items():
    if v >= 40:
        passed[k] = v

grades = []
for v in passed.values():
    if v >= 75:
        grades.append("A")
    else:
        grades.append("B")
```

After — 4 lines

```
names = [n.strip().title()
          for n in row if n.strip()]

book = dict(zip(names, marks))

passed = {k: v for k, v in book.items()
          if v >= 40}

grades = ["A" if v >= 75 else "B"
          for v in passed.values()]
```

- **Filter + transform** strip, title, and drop blanks
- **zip** two lists into one dict
- **if at the end** keep only the passes
- **if/else at the front** one grade per student

Same result, same logic — but a reader now takes in each step at a glance instead of tracing an accumulator.

Practice session — 30 minutes



Predict the output

Filtering, if/else position, set vs dict

8 questions



Spot & fix the bug

Wrong if position, generator, side effects

4 questions



Write the code

Convert loops, filter, invert, flatten, clean

5 tasks

Your notebook

[Day6_Practice_Questions.ipynb](#)

- Write the loop FIRST
- Then convert it
- Raise a hand — I am walking around
- Solutions shared after class

Stuck on a comprehension? Write it as a loop first, then move the pieces. That is not a step backwards — it is how experienced people write the tricky ones too.

Recap

- 1 [expression for item in iterable] builds a list.
- 2 The pieces are the loop's pieces, reordered.
- 3 if at the END filters — the result can be shorter.
- 4 if/else at the FRONT chooses — same length always.
- 5 A bare if at the front is a `SyntaxError`.
- 6 {k: v for ...} builds a dict — the colon decides.
- 7 {expr for ...} builds a set — duplicates dropped.
- 8 (expr for ...) is a generator, NOT a tuple.
- 9 A generator is empty after one pass.
- 10 Two for clauses flatten; nested brackets nest.
- 11 Never use one just for a side effect like `print`.
- 12 If you cannot read it aloud, write the loop.

Homework

Rewrite three loops from Days 2-5 as comprehensions · Build {word: length} for a sentence · Find one comprehension that is WORSE than its loop, and explain why

Next class — Topic 1.7

Classes & objects: defining a class, `__init__`, attributes, methods, and creating objects